Avishkar Project Roadmap

Development of a Financial Decision Support System Using Numerical Methods and Monte Carlo Simulation

A practical one-month roadmap for a college-level Avishkar project. The goal is to build a reliable financial calculator first, then add numerical analysis and Monte Carlo simulation as the main advanced component.

1. Final Scope

The project has three layers: **Basic financial models**, **Intermediate numerical/financial analysis**, and an **Advanced Monte Carlo simulation module**. The project should emphasize mathematical correctness, transparent assumptions, validation, and useful interpretation rather than claiming that similar calculators do not exist.

Layer	Models / features	Priority
Basic	SI, CI, PV, FV, EMI/loan amortization, SIP, inflation/purchasing power	Must have
Intermediate	NPV, IRR, sensitivity analysis, goal-based calculation	If time permits
Advanced	Monte Carlo simulation, outcome distribution, target-reaching probability, parameter controls	Main advanced feature
Interface	Streamlit dashboard, validation, tables, charts, navigation	Must have
Validation	Manual checks, independent checks, normal/boundary/invalid tests	Must have

2. Technology

Python for mathematical logic; **NumPy** for numerical arrays and simulation; **Pandas** for tabular/time-series data; **Matplotlib or Plotly** for graphs; **Streamlit** for the interface; **Git** for version history. AI can assist with small programming tasks and debugging, subject to institutional rules, while mathematical decisions and verification remain the student's responsibility.

3. Project Structure

```
financial_decision_support_system/
■■■ app.py
■■■ models/
■   ■■■ simple_interest.py
■   ■■■ compound_interest.py
■   ■■■ present_value.py
■   ■■■ future_value.py
■   ■■■ emi.py
■   ■■■ sip.py
■   ■■■ inflation.py
■   ■■■ npv.py
■   ■■■ irr.py
■   ■■■ monte_carlo.py
■■■ tests/
■■■ README.md
■■■ requirements.txt
```

4. Four-Week Roadmap

Week 1 — Foundation and Basic Models

- Create the Git repository and project folders.
- Refresh only the Python required: functions, lists/dictionaries, loops, conditionals, imports and basic exceptions.
- Implement SI, CI, PV and FV as independent functions.
- Test every model using hand calculations.
- Start EMI and SIP if the first four models are stable.
- Keep the UI minimal; mathematical correctness comes first.
- Commit each meaningful milestone to Git.

Week 2 — Complete the Calculator and Validate It

- Finish EMI/loan amortization, SIP and inflation/purchasing-power calculations.

- Add input validation and sensible error messages.
- Create amortization and periodic-investment tables.
- Implement NPV.
- If time permits, implement IRR using Bisection or Newton–Raphson after understanding the method.
- Create a test sheet containing normal, zero, boundary and invalid cases.
- Compare selected results with manual calculations or an independent trusted calculator/Excel.

Week 3 — Monte Carlo and Mathematical Analysis

- Learn the probability ideas required by the chosen simulation model.
- Define assumptions: initial investment, contribution pattern, expected return, volatility, time horizon, target and number of simulations.
- Generate random outcomes and repeated investment paths.
- Store final outcomes and calculate summary statistics such as mean, median and selected percentiles.
- Estimate the proportion of simulated outcomes reaching a user-defined target.
- Plot the distribution of final outcomes and representative paths.
- Test controlled cases and document assumptions and limitations.

Week 4 — Integration, UI, Testing and Submission

- Build the Streamlit interface around the already-tested mathematical functions.
- Organize it into Basic Calculators, Investment Analysis and Monte Carlo Simulation.
- Add clear labels, units, defaults, validation messages and explanations.
- Integrate result tables and charts.
- Perform end-to-end testing.
- Freeze major features before the final days; avoid adding untested features at the last minute.
- Prepare screenshots, report, demonstration and judge-question preparation.

5. Mathematical Model Roadmap

Model	Core mathematics	What to demonstrate
Simple Interest	SI = PRT/100	Basic formula and verification
Compound Interest	A = P(1+r/n)^(nt)	Compounding and exponential growth
PV / FV	FV = PV(1+r)^t; PV = FV/(1+r)^t	Time value of money and conventions
EMI	EMI = P r(1+r)^n / ((1+r)^n−1)	Recurring payment plus amortization table
SIP	Periodic contribution accumulation under a stated convention	Contribution timing and growth
Inflation	Future purchasing power using an explicit inflation assumption	Nominal vs real value
NPV	Discount and sum cash flows	Cash-flow timing and discounting
IRR	Solve NPV(r)=0 numerically	Bisection/Newton–Raphson if time permits
Monte Carlo	Repeated random outcomes under explicit assumptions	Distribution, percentiles and target probability

6. Monte Carlo Module — Main Advanced Component

- Inputs: initial investment, optional periodic contribution, time horizon, expected return, volatility, target amount and simulation count.
- Generate random outcomes according to the chosen mathematical model.
- Repeat the process many times to produce a distribution of possible final values.
- Display the distribution instead of only one deterministic forecast.
- Report selected statistics and the estimated probability of reaching the target.
- Visualize the distribution and, optionally, representative simulated paths.
- Clearly state that simulation results depend on assumptions and are not guaranteed future returns.

7. Validation and Testing

- Verify formulas with manual calculations before coding.
- Cross-check representative outputs with Excel or another independent trusted calculation.
- Keep percentage conventions consistent: for example, 5% may be entered by the user while the internal decimal representation is 0.05.
- Test zero and boundary values as well as invalid inputs.
- Use a fixed random seed during Monte Carlo testing when reproducibility is needed.
- After changing a model, rerun previous tests to catch regressions.
- Record important test cases and results for the report.

8. Public Benefit and Uniqueness

Do not claim that no other financial calculator exists. The defensible contribution is the integration and implementation of financial mathematics with probabilistic decision support in one application. The public-use angle is helping users explore financial calculations and understand how assumptions and uncertainty affect possible outcomes. Present the system as a decision-support tool, not as a guaranteed investment adviser.

9. Report Structure

1. Abstract and introduction
2. Problem statement and objectives
3. Existing approach and motivation
4. Mathematical background and derivations
5. Methodology and algorithms

6 System architecture and implementation

7 Monte Carlo assumptions and methodology

8 Results and visualizations

9 Validation and testing

10 Public usefulness

11 Limitations

12 Conclusion and future scope

13 References

10. Future Scope — Not Required for the First Submission

Possible later extensions include portfolio-level simulation, risk measures such as Value at Risk, option-pricing models, Black–Scholes, more sophisticated stochastic models, portfolio optimization and additional numerical methods. Keep these as future scope unless the core application is already complete and tested.

11. Questions to Prepare For

- **Why is this more than a basic calculator?** Explain the integration of financial models with probabilistic decision support.
- **What is new?** Do not claim worldwide novelty. Explain your particular integration, implementation, assumptions and user-oriented analysis.
- **Why Monte Carlo?** Financial outcomes are uncertain; simulation can show a distribution of possible outcomes under stated assumptions.
- **How did you verify it?** Explain manual calculations, independent comparisons and test cases.
- **Can it predict the future?** No. It estimates outcomes under a mathematical model and its assumptions.
- **Why Python?** It supports numerical computation, simulation, visualization and rapid development.

12. Immediate Starting Point

Start with the mathematical engine, not the final UI. Create the Git repository, set up the folders, refresh Python functions, and implement/test Simple Interest. Then move to Compound Interest, PV and FV. The first milestone is a small, correct and testable model. Once the basic models are stable, build the Streamlit interface around them and reserve the advanced development time for Monte Carlo.